

Reviewer Pack — Minimal Order of Affine Algebras Bounds Resolution Proof Siz...

Entry dc61f85e28f0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 01:22:44 UTC

Minimal Order of Affine Algebras Bounds Resolution Proof Size

Entry ID: dc61f85e28f0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 01:22:44 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Affine Algebras) **Field B** (complexity object): Boolean
Function Complexity: Resolution Proof Complexity

Statement:

For a given Boolean formula φ with n variables, the minimal order of an affine algebra over $\text{GF}(2)$ that can represent φ is $\Theta(\log n)$. Specifically, there exists a constant c such that for all $n \geq 1$ and for any boolean formula φ with n variables, the minimal order of an affine algebra representing φ is at most $O(\log n)$.

Rationale (proposer's reasoning):

Affine algebras are closely related to Boolean functions through their ability to represent linear transformations over finite fields. This conjecture suggests that the complexity of a Boolean function in terms of resolution proof size might be directly linked to the structure and order of an associated affine algebra, offering a novel perspective on resolution-based complexity theory.

Taxonomy category: affine_algebra_resproof (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 025ef0deab7eb0ae

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The minimal order of an affine algebra representing a Boolean formula φ with n variables will be at most $O(\log n)$ if and only if the Pearson correlation coefficient between the order of the affine algebras and $\log(n)$ is greater than or equal to 0.8, across all generated formulas with $n \leq 40$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "affine algebra" AND "Boolean function complexity" AND "resolution proof size" - "minimal order" IN AFFINE ALGEBRA AND resolution proof complexity" - "GF(2) affine algebra" AND boolean formula AND $\Theta(\log n)$

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_formula(n):
        if n == 1:
            return 'x'
        else:
            op = random.choice(['&', '|'])
            left = generate_boolean_formula(random.randint(1, n-1))
            right = generate_boolean_formula(random.randint(1, n-1))
            return f'({left} {op} {right})'

    def construct_affine_algebra(formula):
        if formula == 'x':
            return 1
        elif '&' in formula:
            left, _, right = formula.split('&', 1)
            order_left = construct_affine_algebra(left.strip())
            order_right = construct_affine_algebra(right.strip())
            return max(order_left, order_right) + 1
        elif '|' in formula:
            left, _, right = formula.split('|', 1)
            order_left = construct_affine_algebra(left.strip())
            order_right = construct_affine_algebra(right.strip())
            return max(order_left, order_right) + 1
        else:
```

```

        raise ValueError("Invalid boolean formula")

n_max = 40
instances_tested = 0
total_order = 0

for n in range(5, n_max + 1):
    formula = generate_boolean_formula(n)
    order = construct_affine_algebra(formula)
    total_order += order
    instances_tested += 1

mean_order = total_order / instances_tested
conjecture_holds = False
counterexample = ""

if instances_tested >= 30:
    log_n_values = [math.log(n, 2) for n in range(5, n_max + 1)]
    correlation_coefficient = calculate_correlation(log_n_values, [order for order in range(5,
    if correlation_coefficient >= 0.8:
        conjecture_holds = True
    else:
        counterexample = f"Correlation coefficient {correlation_coefficient} < 0.8"

return {
    "metric_name": "mean_order",
    "metric_value": mean_order,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

def calculate_correlation(x, y):
    if len(x) != len(y):
        raise ValueError("x and y must have the same length")

    n = len(x)
    mean_x = sum(x) / n
    mean_y = sum(y) / n

    numerator = sum((xi - mean_x) * (yi - mean_y) for xi, yi in zip(x, y))
    denominator = math.sqrt(sum((xi - mean_x)**2 for xi in x)) * math.sqrt(sum((yi - mean_y)**2 fo

    return numerator / denominator

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```

```

mean_value = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Correlation coefficient < 0.8\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bf3edfaf.py", line 96, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bf3edfaf.py", line 52, in run_trial
    order = construct_affine_algebra(formula)
            ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bf3edfaf.py", line 34, in construct_affine_algebra
    left, _, right = formula.split('&', 1)
    ~~~~~^
ValueError: not enough values to unpack (expected 3, got 2)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to provide a result for the conjecture. | next: Investigate and fix the crash in the test code to allow for proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	22585
2	preregistration	ollama_remote	glm4:latest	0	0	14110
3	novelty	ollama_remote	glm4:latest	0	0	8621

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	glm4:latest	0	0	19297
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15316
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	30088
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11536
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11675
9	judge	ollama_remote	glm4:latest	0	0	9704

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 142932 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/dc61f85e28f0.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/dc61f85e28f0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/dc61f85e28f0.tar.gz` (if generated)